

**IC TRAINING CENTER VIET NAM
FUNDAMENTAL IC DESIGN AND VERIFICATION COURSE**



**FINAL PROJECT
TIMER IP DESIGN SPECIFICATION**

Student: Do Luong Minh Hoang

Class: ICTC35

Instructor: Tran Minh Duc

Contents

1. Overview.....	4
1.1. Introduction.....	4
1.2. Main features.....	4
1.3. Block diagram.....	4
1.4. Interface signals.....	5
2. Register Specification.....	14
2.1. Register Summary	15
2.2. Timer Control Register (TCR).....	15
2.3. Timer Data Register 0 (TDR0).....	18
2.4. Timer Data Register 0 (TDR1).....	18
2.5. Timer Compare Register 0 (TCMP0).....	19
2.6. Timer Compare Register 1 (TCMP1).....	19
2.7. Timer Interrupt Enable Register (TIER).....	20
2.8. Timer Interrupt Status Register (TISR).....	21
2.9. Timer Halt Control Status Register (THCSR).....	22
3. Functional Description.....	23
3.1. Register.....	23
3.1.1. Timer Control Register (0x00).....	23
3.1.2. Byte Strobe Logic.....	25
3.1.3. Timer Data Register 0/1 (0x04 & 0x08).....	27
3.1.4. Negative Edge Detector.....	30
3.1.5. Timer Compare Register 0/1 (0x0C & 0x10).....	31
3.1.6. Timer Interrupt Enable Register (0x14).....	33
3.1.7. Timer Interrupt Status Register (0x18).....	35
3.1.8. Timer Halt Control Status Register (0x1C).....	37
3.1.9. Read Logic.....	39
3.2. Counter.....	40
3.2.1. Clock Tick Enable Logic.....	42
3.3. APB Slave.....	43
3.3.1. Wait State Logic.....	45

4. History.....	47
-----------------	----

LIST OF TABLES

Table 1: Interface signals of timer IP	6
Table 2: Interface signals of APB Slave IP	8
Table 3: Interface signals of Register IP	10
Table 4: Interface signals of 64-bit counter IP	13
Table 5: Register Summary	15
Table 6: Timer Control Register - TCR	16
Table 7: Timer Data Register 0 - TDR0	19
Table 8: Timer Data Register 1 - TDR1	19
Table 9: Timer Compare Register 0 - TCMP0	20
Table 10: Timer Compare Register 1 - TCMP1	20
Table 11: Timer Interrupt Enable Register - TIER	21
Table 12: Timer Interrupt Status Register - TISR	22
Table 13: Timer Halt Control Status Register - THCSR	23

LIST OF FIGURES

Figure 1: Block diagram of Timer IP	5
Figure 2: Waveform for Timer Control Register	24
Figure 3: Logic Diagram of Timer Control Register	25
Figure 4: Waveform for Byte Strobe Logic	26
Figure 5: Logic Diagram of Byte Strobe Logic	27
Figure 6: Waveform for Timer Data Register	28
Figure 7: Logic Diagram of Timer Data Register 0/1	29
Figure 8: Waveform for Negative Edge Detector	32
Figure 9: Logic Diagram of Negative Edge Detector	32
Figure 10: Waveform for Timer Compare Register	33
Figure 11: Logic Diagram of Timer Compare Register	34
Figure 12: Waveform for Timer Interrupt Enable Register	35
Figure 13: Logic Diagram of Timer Interrupt Enable Register	36
Figure 14: Waveform for Timer Interrupt Status Register	37

Figure 15: Logic Diagram of Timer Interrupt Status Register	38
Figure 16: Waveform for Timer Halt Control Status Register	39
Figure 17: Logic Diagram of Timer Halt Control Status Register	39
Figure 18: Waveform for Read Logic	40
Figure 19: Logic Diagram of Read Logic	41
Figure 20: Waveform for Counter	41
Figure 21: Logic Diagram of Counter	42
Figure 22: Waveform for Clock Tick Enable Logic	43
Figure 23: Logic Diagram of Clock Tick Enable Logic	43
Figure 24: Waveform for APB Slave	44
Figure 25: Logic Diagram of APB Slave	45
Figure 26: Waveform for Wait State Logic	46
Figure 27: Logic Diagram of Wait State Logic	47

1. Overview:

1.1 Introduction:

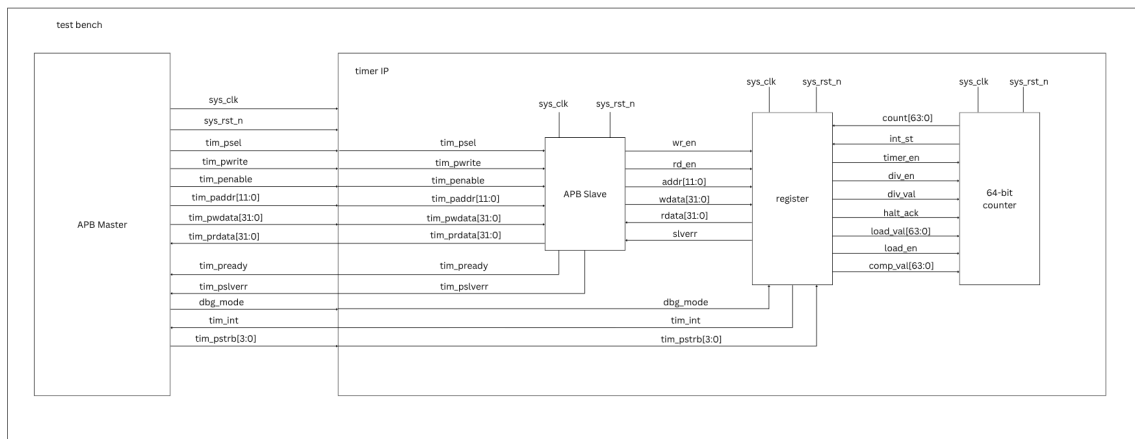
- This timer uses a 64-bit counter, which is configured by a set of registers with 12-bit addresses. The register set is configured via APB bus (IP is APB Slave). The timer module allows developers to generate interrupts, test, and debug the system during development and testing.

1.2 Main Features:

- 64-bit counter.
- 12-bit address register set.
- Register set is configured via APB bus (IP is APB Slave).
- Support APB 32-bit transfers with wait state.
- Support error handling.
- Support byte access.
- Support halt in debug mode.
- Timer uses low active async reset.
- Counter can be counted based on system clock or divided up to 256.
- Support hardware - maskable - level timer interrupt (can be enabled or disabled).

1.3 Block Diagram:

(Figure 1: Block diagram of Timer IP)



- **APB Slave** is the interface between the **master** and the **register** set. It performs a R/W access to the register set.
- The **register set** stores configuration bits of the **counter**. It can enable the counter, adjust the counter's counting speed, store count values, enable/disable and trigger timer interrupt, as well as trigger halting of the counter in debug mode.
- The **64-bit counter** counts normally in normal mode, during interrupt, and stops counting when there is a halt request.
- The **timer** is the DUT that connects all sub modules together.
- The **master** is part of the testbench and is the stimulus generator for the timer DUT.

1.4 Interface Signals:

(Table 1: Interface signals of timer IP)

Signals	Width	Direction	Description
sys_clk	1	Input	IP clock
sys_rst_n	1	Input	Low-active async reset
tim_psel	1	Input	Slave select signal 0: this slave is not chosen for transfer 1: set up transfer for the chosen slave
tim_pwrite	1	Input	R/W access signal 0: a read access is requested 1: a write access is requested
tim_penable	1	Input	R/W access enable 0: access is enabled, transfer

			can begin anytime 1: access is denied, no transfer is allowed
tim_paddr	12	Input	Access bus
tim_pwdata	32	Input	Write bus
tim_prdata	32	Output	Read bus
tim_pstrb	4	Input	Byte access support tim_pstrb[0] corresponds to byte range 0 → 7 of tim_pwdata tim_pstrb[1] corresponds to byte range 8 → 15 of tim_pwdata tim_pstrb[2] corresponds to byte range 16 → 23 of tim_pwdata tim_pstrb[3] corresponds to byte range 24 → 31 of tim_pwdata
tim_pready	1	Output	Transfer ready signal 0: transfer is accepted at the next pos clk 1: transfer is not accepted
tim_pslverr	1	Output	Error handling signal 0: successful

			transfer 1: error occurred during transfer
tim_int	1	Output	indicates pending timer interrupt 0: timer interrupt is not enabled or/and interrupt trigger condition has not occurred 1: timer interrupt is enabled and interrupt trigger condition has occurred
dbg_mode	1	Input	Debug mode bit 0: timer is not in debug mode 1: timer is in debug mode, counter is halted.

(Table 2: Interface signals of APB Slave IP)

Signals	Width	Direction	Description
sys_clk	1	Input	IP clock
sys_rst_n	1	Input	Low-active async reset
tim_psel	1	Input	Slave select signal 0: this slave is not chosen for transfer 1: set up transfer for the chosen slave
tim_pwrite	1	Input	R/W access signal

			0: a read access is requested 1: a write access is requested
tim_penable	1	Input	R/W access enable 0: access is enabled, transfer can begin anytime 1: access is denied, no transfer is allowed
tim_paddr	12	Input	Access bus
tim_pwdata	32	Input	Write bus
tim_prdata	32	Output	Read bus
tim_pready	1	Output	Transfer ready signal 0: transfer is accepted at the next pos clk 1: transfer is not accepted
slverr	1	Input	Error handling signal received from register 0: successful transfer 1: error occurred during transfer
tim_pslverr	1	Output	Error handling signal 0: successful transfer 1: error occurred during transfer

wr_en	1	Output	write enable bit 0: prohibits write access to register 1: permits write access to register
rd_en	1	Output	read enable bit 0: prohibits read access to register 1: permits read access to register
addr	12	Output	address bus between APB slave and register
wdata	32	Output	write bus between APB slave and register
rdata	32	Input	read bus between APB slave and register

(Table 3: Interface signals of Register IP)

Signals	Width	Direction	Description
sys_clk	1	Input	IP clock
sys_rst_n	1	Input	Low-active async reset
wr_en	1	Input	write enable bit 0: prohibits write access to register 1: permits write access to register
rd_en	1	Input	read enable bit 0: prohibits read

			access to register 1: permits read access to register
addr	12	Input	address bus between APB slave and register
wdata	32	Input	write bus between APB slave and register
rdata	32	Output	read bus between APB slave and register
count	64	Input	count value of 64-bit counter
timer_en	1	Output	timer enable 0: Disabled. Counter does not count 1: Enabled. Counter starts counting
div_en	1	Output	Counter control mode enable 0: Disabled. Counter counts with normal speed based on system clock 1: Enabled. The counting speed of counter is controlled based on div_val
div_val	4	Output	Counter control mode setting
tim_int	1	Output	indicates pending timer interrupt

			0: timer interrupt is not enabled or/and interrupt trigger condition has not occurred 1: timer interrupt is enabled and interrupt trigger condition has occurred
dbg_mode	1	Input	Debug mode bit 0: timer is not in debug mode 1: timer is in debug mode, counter is halted.
halt_ack	1	Output	Timer halt acknowledgement 0: timer is not halted 1: timer is halted
int_st	1	Input	Timer interrupt trigger condition status bit (interrupt pending bit) 0: the interrupt trigger condition does not occur. 1: the interrupt trigger condition occurred.
tim_pstrb	4	Input	Byte access support tim_pstrb[0] corresponds to byte

			range 0 → 7 of tim_pwdata tim_pstrb[1] corresponds to byte range 8 → 15 of tim_pwdata tim_pstrb[2] corresponds to byte range 16 → 23 of tim_pwdata tim_pstrb[3] corresponds to byte range 24 → 31 of tim_pwdata
slverr	1	Output	Error handling signal 0: successful transfer 1: error occurred during transfer

(Table 4: Interface signals of 64-bit counter IP)

Signals	Width	Direction	Description
sys_clk	1	Input	IP clock
sys_rst_n	1	Input	Low-active async reset
count	64	Output	count value of 64-bit counter
comp_val	64	Input	compare value of 64-bit counter. When count value is equal to compare value, the interrupt pending bit is set.

timer_en	1	Input	timer enable 0: Disabled. Counter does not count 1: Enabled. Counter starts counting
div_en	1	Input	Counter control mode enable 0: Disabled. Counter counts with normal speed based on system clock 1: Enabled. The counting speed of counter is controlled based on div_val
div_val	4	Input	Counter control mode setting
halt_ack	1	Input	Timer halt acknowledgement 0: timer is not halted 1: timer is halted
int_st	1	Output	Timer interrupt trigger condition status bit (interrupt pending bit) 0: the interrupt trigger condition does not occur. 1: the interrupt trigger condition occurred.

load_val	64	Input	loaded counter's value received from TDR0/1
load_en	1	Input	flag signal to indicate whether counter can accept load_val from TDR0/1 0: counter does not accept loaded value from TDR0/1 1: counter accepts loaded value from TDR0/1

2. Register Specification:

2.1 Register Summary:

(Table 5: Register Summary)

Address	Abbreviation	Register name
0x00	TCR	Timer Control Register
0x04	TDR0	Timer Data Register 0
0x08	TDR1	Timer Data Register 1
0x0C	TCMP0	Timer Compare Register 0
0x10	TCMP1	Timer Compare Register 1
0x14	TIER	Timer Interrupt Enable Register
0x18	TISR	Timer Interrupt Status Register
0x1C	THCSR	Timer Halt Control Status Register
Others	Reserved	

2.2 Timer Control Register (TCR):

(Table 6: Timer Control Register - TCR)

Bits	Name	Type	Default Value	Description
31:12	Reserved	-	20'h0	Reserved
11:8	div_val	RW	4'h0001	Counter control mode setting: • 4'b0000: Counting speed is not divided • 4'b0001: Counting speed is divided by 2 (default) • 4'b0010: Counting speed is divided by 4 • 4'b0011: Counting speed is divided by 8 • 4'b0100: Counting speed is divided by 16 • 4'b0101: Counting speed is divided by 32 • 4'b0110: Counting speed is divided by 64

				• 4'b0111: Counting speed is divided by 128 • 4'b1000: Counting speed is divided by 256 • Others: reserved, (*)prohibit settings. When setting the prohibit value, div_val is not changed. Note: user must not change div_val while timer_en is High (*): add hardware logic to ensure div_val is prohibited to change when timer_en is High. Access is error response in this case. (*)access is "error response" when setting prohibit value to div_val
--	--	--	--	---

7:2	Reserved	RO	6'h0	Reserved
1	div_en	RW	1'b0	Counter control mode enable. • 0: Disabled. Counter counts with normal speed based on system clock • 1: Enabled. The counting speed of counter is controlled based on div_val Note: user must not change div_en while timer_en is High (*): add hardware logic to ensure div_en is prohibited to change when timer_en is High. Access is error response in this case.
0	timer_en	RW	1'b0	Timer enable • 0: Disabled. Counter does

				not count. • 1: Enabled. Counter starts counting. (*) timer_en changes from H->L will initialize the TDR0/1 to their initial value
--	--	--	--	---

2.3 Timer Data Register 0 (TDR0):

(Table 7: Timer Data Register 0 - TDR0)

Bits	Name	Type	Default Value	Description
31:0	TDR0	RW	32'h0000_0000	Lower 32-bit of 64-bit counter. Note: value of this register is cleared to initial value when timer_en changes from H->L.

2.4 Timer Data Register 1 (TDR1):

(Table 8: Timer Data Register 1 - TDR1)

Bits	Name	Type	Default Value	Description
31:0	TDR1	RW	32'h0000_0000	Upper 32-bit of 64-bit counter.

				Note: value of this register is cleared to initial value when timer_en changes from H->L.
--	--	--	--	---

2.5 Timer Compare Register 0 (TCMP0):

(Table 9: Timer Compare Register 0 - TCMP0)

Bits	Name	Type	Default Value	Description
31:0	TCMP0	RW	32'hFFFFFF_F	Lower 32-bit of 64-bit compare value. Interrupt pending bit is asserted when counter value is equal to compare value.

2.6 Timer Compare Register 1 (TCMP1):

(Table 10: Timer Compare Register 1 - TCMP1)

Bits	Name	Type	Default Value	Description
31:0	TCMP1	RW	32'hFFFFFF_F	Upper 32-bit of 64-bit compare value. Interrupt pending bit is asserted when

				counter value is equal to compare value.
--	--	--	--	--

2.7 Timer Interrupt Enable Register (TIER):

(Table 11: Timer Interrupt Enable Register - TIER)

Bits	Name	Type	Default Value	Description
31:1	Reserved	RO	31'h0	Reserved
0	int_en	R/W	1'b0	Timer interrupt enable 0: Timer interrupt is disabled. 1: Timer interrupt is enabled. When this bit is 0, no timer interrupt is output. When this bit is 1, timer interrupt pending bit can be output (interrupt pending bit is set when counter value is equal to compare value). Clearing this bit to 0 while

				interrupt is asserting will mask the interrupt to 0 but does not affect the interrupt pending bit (TISR.int_st bit).
--	--	--	--	--

2.8 Timer Interrupt Status Register (TISR):

(Table 12: Timer Interrupt Status Register - TISR)

Bits	Name	Type	Default Value	Description
31:1	Reserved	RO	31'h0	Reserved
0	int_st	RW1C	1'b0	Timer interrupt trigger condition status bit (interrupt pending bit) 0: the interrupt trigger condition does not occur. 1: the interrupt trigger condition occurred. Write 1 when this bit is 1 to clear it to 0. Write 0 when this bit is 1 has no effect

				Write to this bit when it is 0 has no effect. Note1: When interrupt trigger condition occurred (counter reached compare value), counter continues to count normally. Note2: the clearance condition has highest priority.
--	--	--	--	--

2.9 Timer Halt Control Status Register (THCSR):

(Table 13: Timer Halt Control Status Register - THCSR)

Bits	Name	Type	Default Value	Description
31:2	Reserved	RO	30'h0	Reserved
1	halt_ack	RO	1'b0	Timer halt acknowledge 0: timer is NOT halted 1: timer is halted Timer accepts the halt request only in debug mode, indicates by

				dbg_mode input signal
0	halt_req	RW	1'b0	Timer halt request 0: no halt req. 1: timer is requested to halt.

3. Functional Description:

3.1. Register:

The register block is the configuration and status store of the timer. It decodes the 12-bit `addr` from the APB Slave, applies per-byte write masking (`tim_pstrb`), holds the eight architectural registers (`TCR`, `TDR0/1`, `TCMP0/1`, `TIER`, `TISR`, `THCSR`), drives the counter control outputs (`timer_en`, `div_en`, `div_val`, `comp_val`, `load_val`, `load_en`), generates the maskable interrupt `tim_int`, the halt acknowledge `halt_ack`, and the slave error `slverr`, and returns read data through the read mux. All flops use a low-active asynchronous reset.

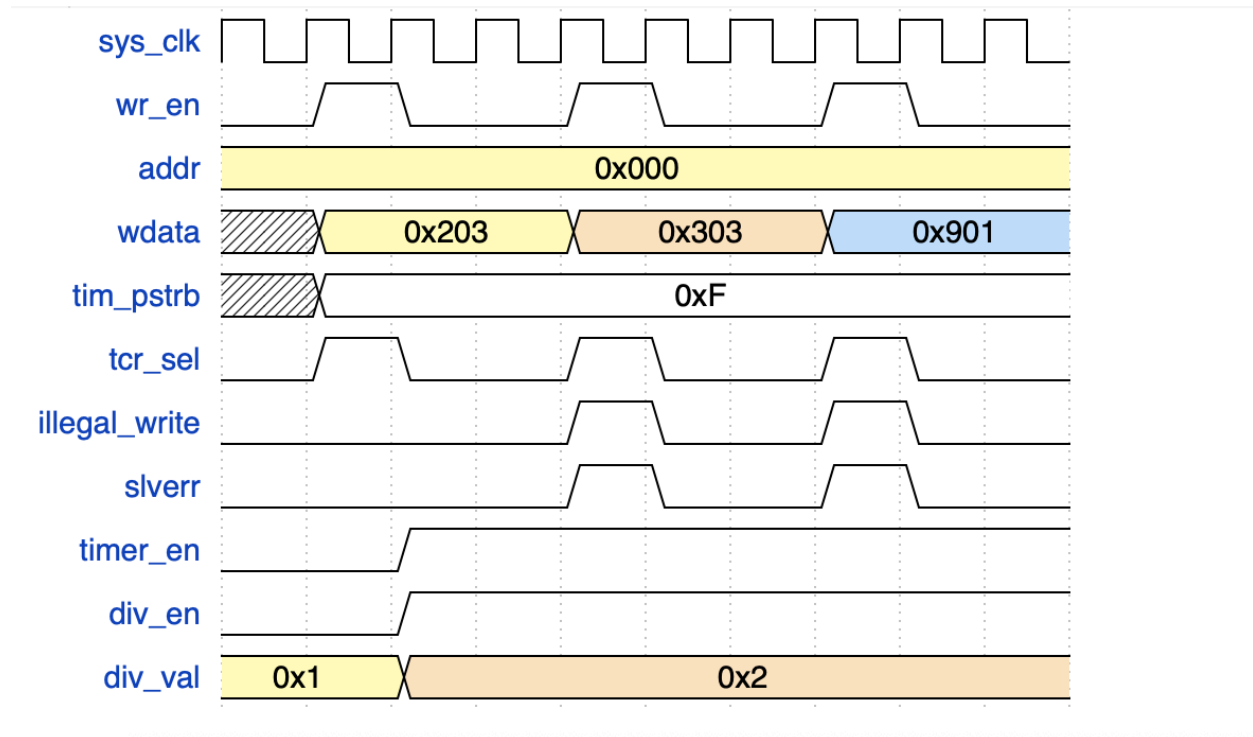
3.1.1. Timer Control Register (0x00):

Module Description: `TCR` holds three live control fields: `timer_en` (bit 0), `div_en` (bit 1) and `div_val` (bits 11:8); all other bits are reserved. Reset value is {20'h0, 4'h1, 6'h0, 1'b0, 1'b0} (`div_val` = 1, `div_en` = 0, `timer_en` = 0). The register adds two guard conditions that produce an **error response** and block the write:

- `prohibit_div` — the new `div_val` is greater than 4'b1000 (a reserved code).
- `illegal_config` — an attempt to change `div_val` or `div_en` while `timer_en` is already High.

When either fires, `illegal_write` = 1, `slverr` is asserted, and the register keeps its current value. Only `tcr_sel && !illegal_write` lets the new fields through.

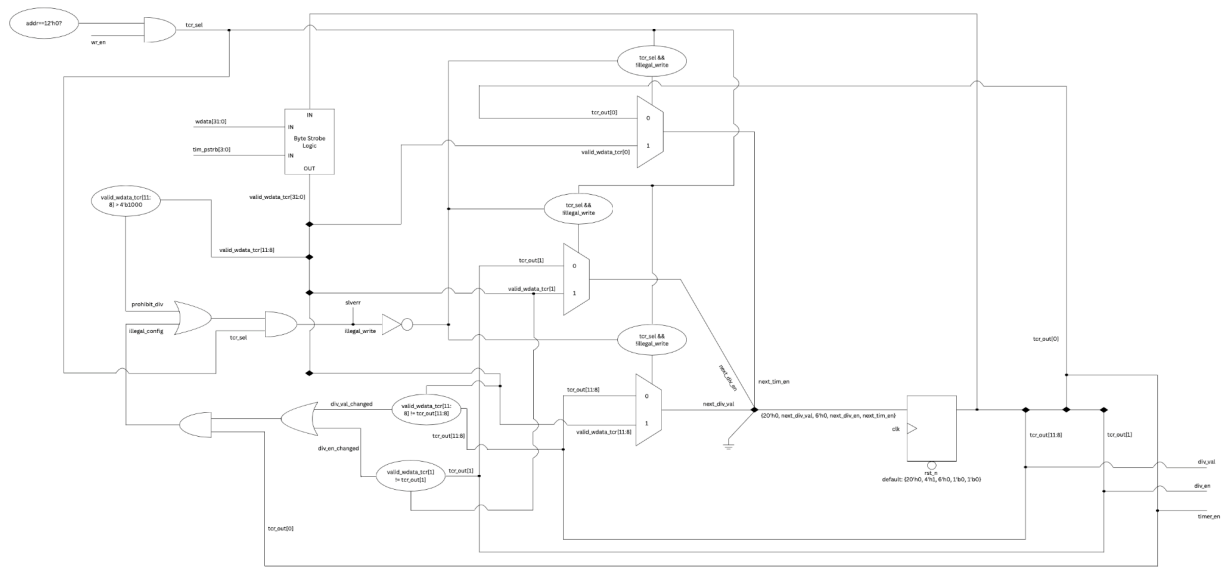
(Figure 2: Waveform for Timer Control Register)



Explain Waveform:

- **Write 1 (legal):** `wdata = 0x0000_0203` sets `timer_en = 1`, `div_en = 1`, `div_val = 2` while `timer_en` was Low. The write is accepted; one clock later `timer_en`, `div_en` and `div_val` update, `slverr` stays Low.
- **Write 2 (illegal_config):** `wdata = 0x0000_0303` tries to change `div_val` (2 → 3) while `timer_en = 1`. `illegal_write` and `slverr` assert in the same cycle as `wr_en`; the register is unchanged.
- **Write 3 (prohibit_div):** `wdata = 0x0000_0901` requests `div_val = 9` (> 8). `prohibit_div` blocks it; `slverr` asserts, fields hold.

(Figure 3: Logic Diagram of Timer Control Register)

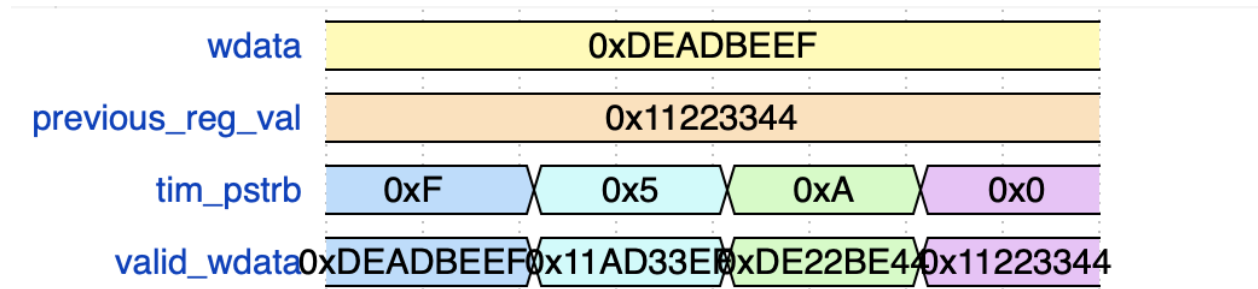


Explain Logic Diagram: `addr == 12'h0` AND `wr_en` produces `tcr_sel`. The byte-strobe block merges `wdata` with the current `tcr_out` to form `valid_wdata_tcr`. Two comparators detect `div_val_changed` and `div_en_changed`; ORed and ANDed with `tcr_out[0]` (`timer_en`) they give `illegal_config`, while `valid_wdata_tcr[11:8] > 4'b1000` gives `prohibit_div`. `illegal_write = tcr_sel && (prohibit_div | illegal_config)` drives `slverr` and the select line of the three field muxes: each field updates from `valid_wdata_tcr` only when `tcr_sel && !illegal_write`, otherwise it recirculates the old value. The packed `{20'h0, next_div_val, 6'h0, next_div_en, next_tim_en}` is registered on `sys_clk`, async-reset to the default.

3.1.2. Byte Strobe Logic:

Module Description: A combinational per-byte write-merge used by every writable register. For each of the four byte lanes, a 2:1 mux selects the incoming `wdata` byte when its strobe bit is set, or recirculates the corresponding byte of the current register value (`previous_reg_val`) when it is clear. This supports byte-granular APB writes without disturbing untouched bytes.

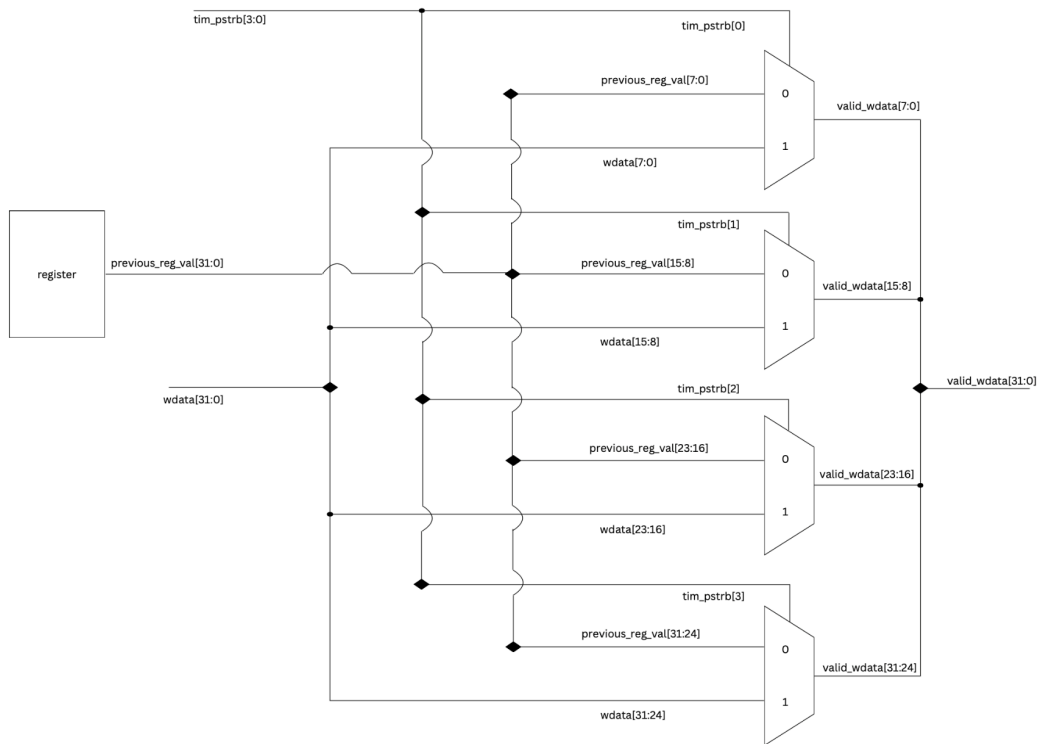
(Figure 4: Waveform for Byte Strobe Logic)



Explain Waveform: With wdata = 0xDEADBEEF and previous_reg_val = 0x11223344:

- pstrb = 0xF → all four lanes take wdata → 0xDEADBEEF.
- pstrb = 0x5 (0b0101) → lanes 0,2 from wdata, lanes 1,3 kept → 0x11AD33EF.
- pstrb = 0xA (0b1010) → lanes 1,3 from wdata, lanes 0,2 kept → 0xDE22BE44.
- pstrb = 0x0 → nothing written → 0x11223344 (unchanged).

(Figure 5: Logic Diagram of Byte Strobe Logic)

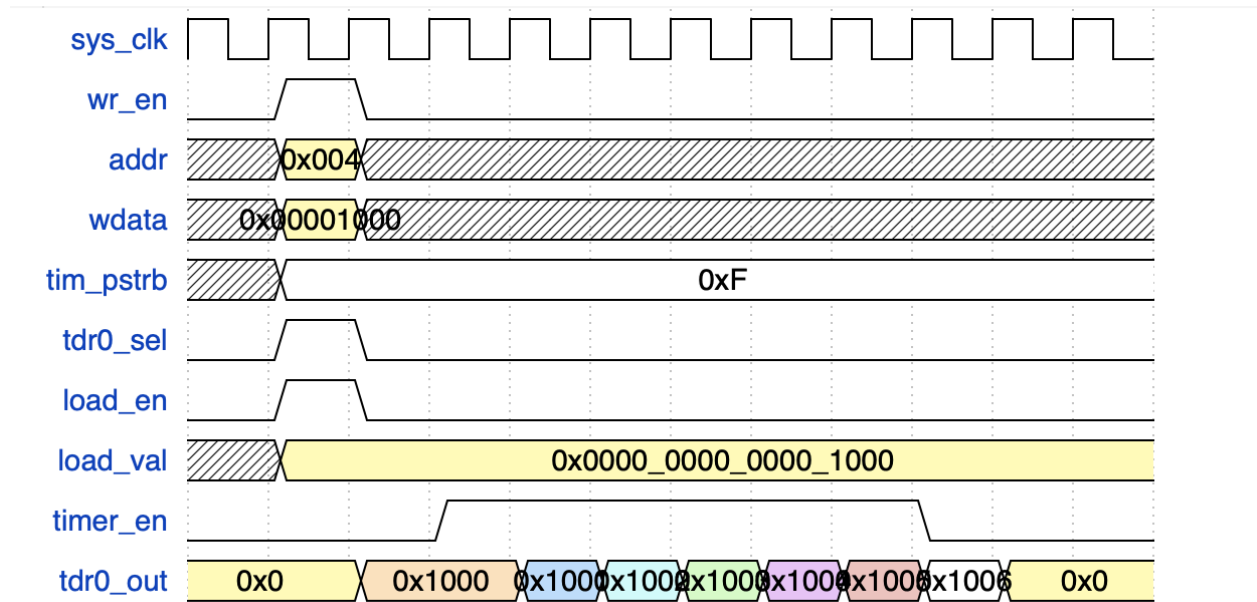


Explain Logic Diagram: `wdata[31:0]` and `previous_reg_val[31:0]` are split into four byte slices. Each slice feeds a mux whose select is the matching `tim_pstrb[i]` (1 → `wdata` byte, 0 → previous byte). The four mux outputs concatenate to `valid_wdata[31:0]`, which is the masked write value handed to the owning register. The "register" block on the left is the source of `previous_reg_val` (the register's own current contents).

3.1.3. Timer Data Register 0/1 (0x04 & 0x08):

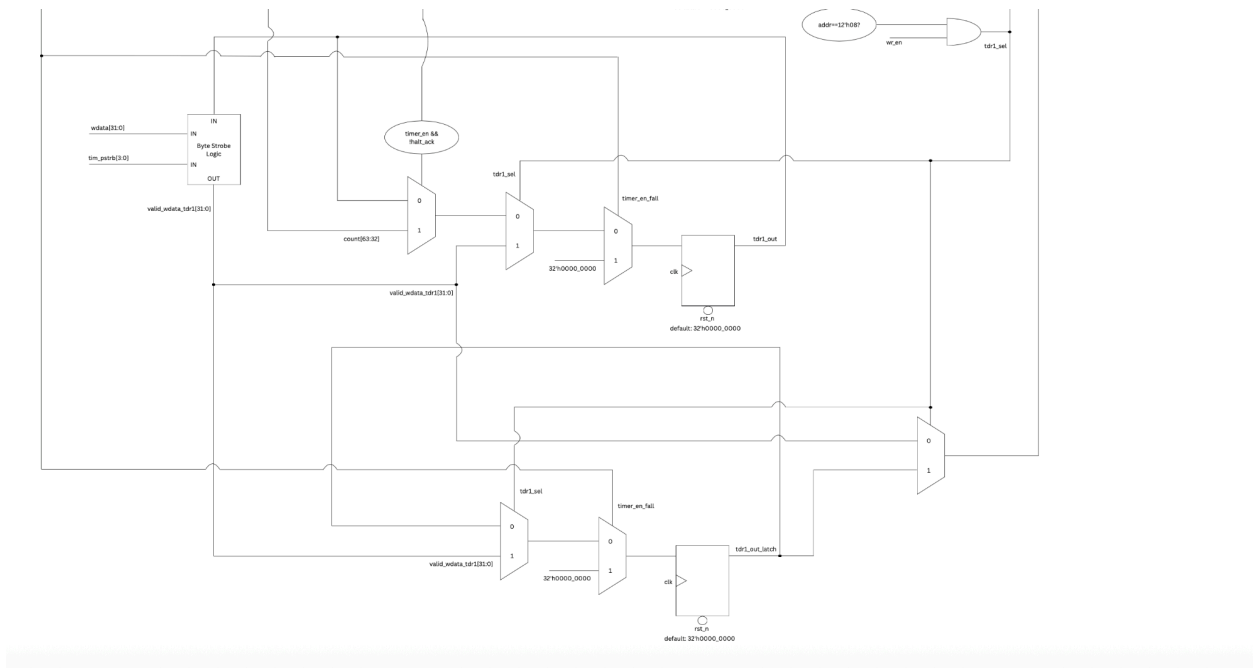
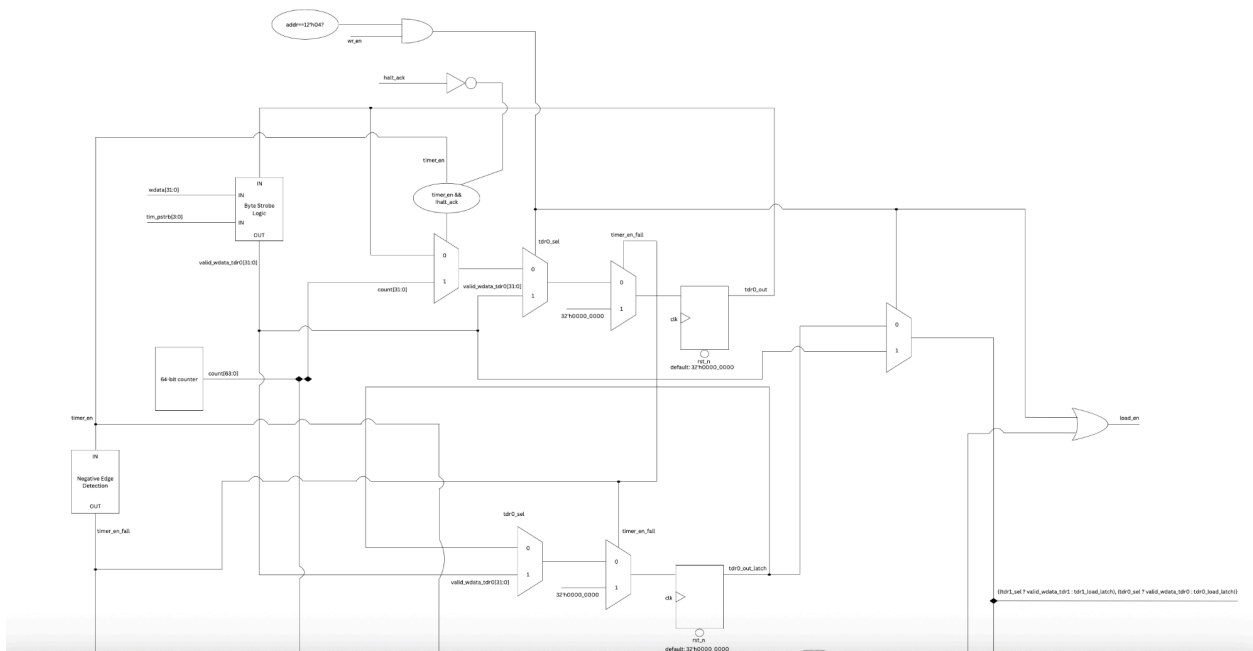
Module Description: TDR0/TDR1 are the lower/upper 32-bit windows of the 64-bit counter and serve two roles. (1) **Load:** a write to either register loads a start value into the counter via `load_val/load_en`. A per-half load-latch (`tldr0/1_load_latch`) preserves the value written to one half so that writing the other half later still presents a coherent 64-bit `load_val`. (2) **Read-back / mirror:** while the timer runs (`timer_en` && `!halt_ack`) and no write is in progress, the register continuously samples the live counter so reads return the current count. On a `timer_en` falling edge both halves clear to 0 (initial value).

(Figure 6: Waveform for Timer Data Register)



Explain Waveform: A write to 0x04 (tdr0_sel) asserts load_en and presents load_val = 0x..._1000; tdr0_out registers the written value one clock later. Once timer_en goes High, tdr0_out tracks the live counter every clock (0x1001, 0x1002, ...). When timer_en falls, the negative-edge detector (3.1.4) fires and tdr0_out is cleared to 0 the following clock. (TDR1 / address 0x08 behaves identically on the upper word.)

(Figure 7: Logic Diagram of Timer Data Register 0/1)



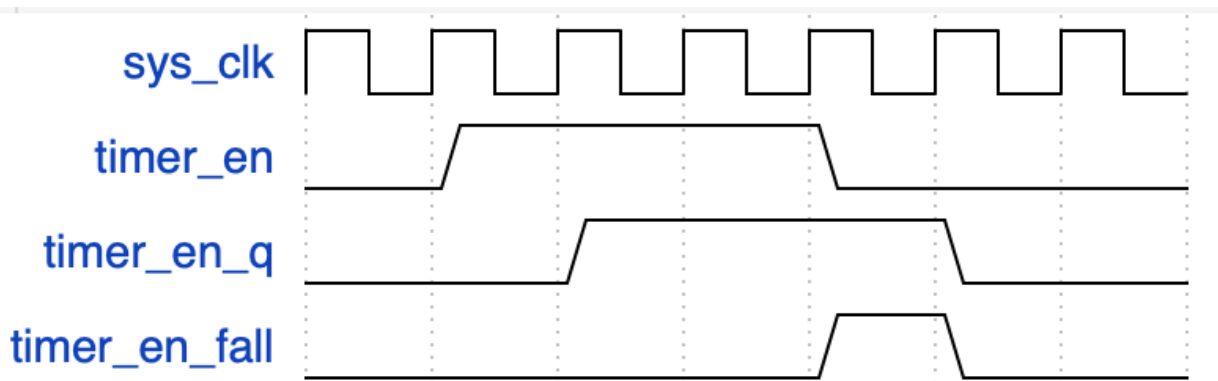
Explain Logic Diagram: Each half is a flop with a 3-way priority input:
timer_en_fall $\rightarrow$ 32'h0; else tdrX_sel $\rightarrow$ valid_wdata_tdrX; else (timer_en
&& !halt_ack) $\rightarrow$ count[31:0] / count[63:32]. A parallel load-latch flop
captures valid_wdata_tdrX on a write and is also cleared by timer_en_fall.
load_en = tdr0_sel | tdr1_sel, and load_val = {(tdr1_sel ?
valid_wdata_tdr1 : tdr1_load_latch), (tdr0_sel ?

`valid_wdata_tdr0 : tdr0_load_latch)` }so the half being written uses the fresh masked data while the other half uses its latch. The Negative Edge Detection block supplies `timer_en_fall`; two Byte Strobe blocks supply the masked write data.

3.1.4. Negative Edge Detector:

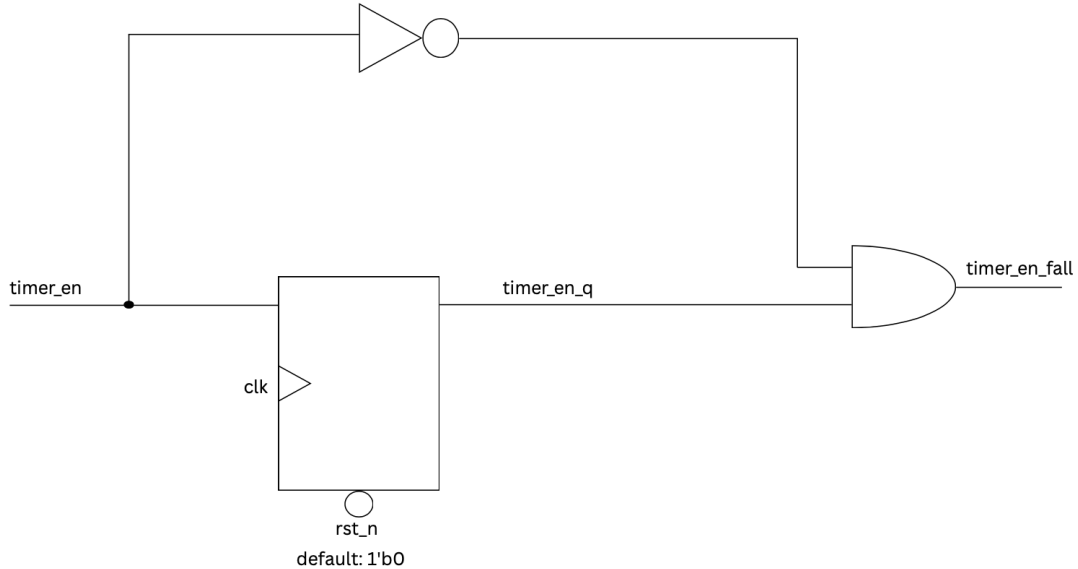
Module Description: Generates a one-cycle pulse `timer_en_fall` on the High→Low transition of `timer_en`. Used to clear TDR0/1 when the timer is disabled.

(Figure 8: Waveform for Negative Edge Detector)



Explain Waveform: `timer_en_q` is `timer_en` delayed one clock. When `timer_en` is Low but `timer_en_q` is still High (the cycle right after the fall), `timer_en_fall` pulses for exactly one clock.

(Figure 9: Logic Diagram of Negative Edge Detector)

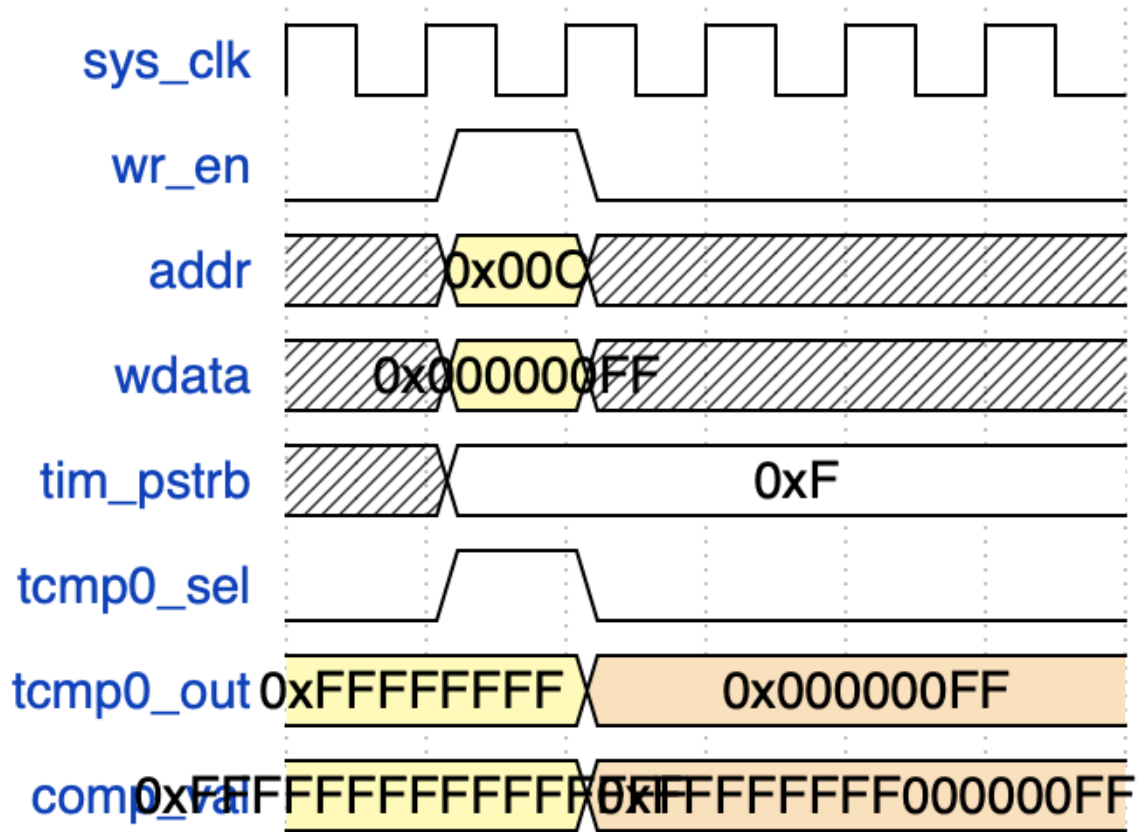


Explain Logic Diagram: A single D flip-flop (async reset, default `1'b0`) produces **timer_en_q**. $\text{timer_en_fall} = (\sim\text{timer_en}) \& \text{timer_en_q}$ — the inverter on **timer_en** feeds an AND gate with **timer_en_q**. The pulse is one clock wide because **timer_en_q** follows **timer_en** Low on the next edge.

3.1.5. Timer Compare Register 0/1 (0x0C & 0x10):

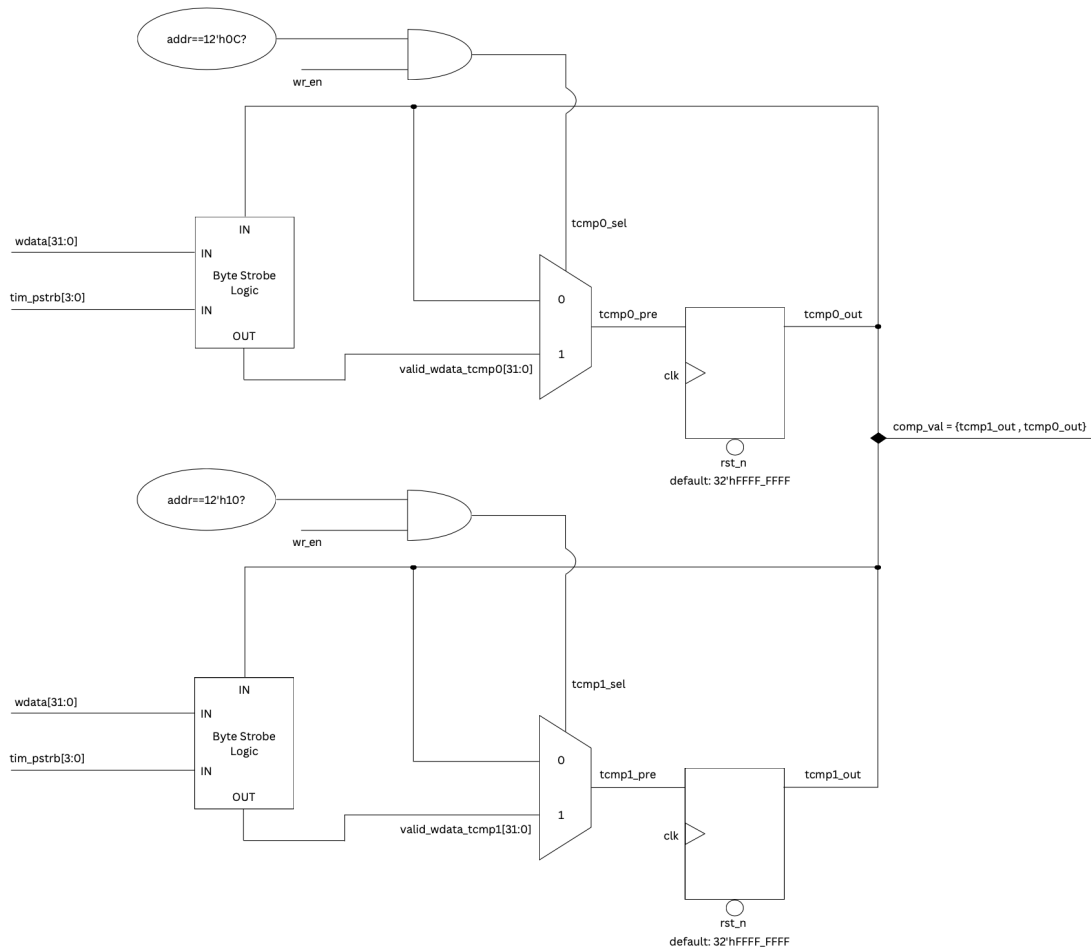
Module Description: TCMP0/TCMP1 hold the lower/upper 32 bits of the 64-bit compare value. `comp_val = {tcmp1_out, tcmp0_out}` is fed to the counter; when the count equals `comp_val`, the interrupt pending bit is set. Both registers reset to `0xFFFF_FFFF` and are byte-writable.

(Figure 10: Waveform for Timer Compare Register)



Explain Waveform: A write to 0x0C with wdata = 0x0000_00FF and all strobes set updates tcmp0_out one clock later from the reset default 0xFFFF_FFFF to 0x0000_00FF; comp_val reflects the change in its lower word. TCMP1 (0x10) drives the upper word the same way.

(Figure 11: Logic Diagram of Timer Compare Register 0/1)

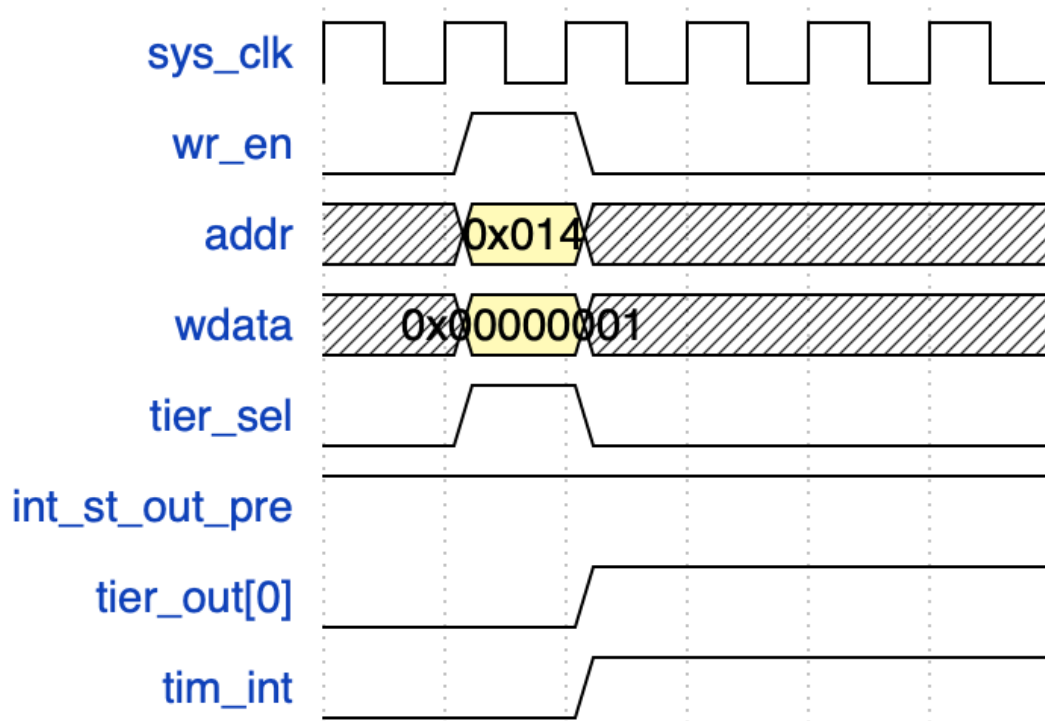


Explain Logic Diagram: `addr == 12'h0C AND wr_en` → `tcmp0_sel`; a byte-strobe block forms `valid_wdata_tcmp0`. The mux `tcmp0_pre = tcmp0_sel ? valid_wdata_tcmp0 : tcmp0_sel` selects new-vs-hold, registered each clock (async reset `0xFFFF_FFFF`). TCMPI is identical at `0x10`. The two outputs concatenate to `comp_val = {tcmp1_out, tcmp0_out}`.

3.1.6. Timer Interrupt Enable Register (0x14):

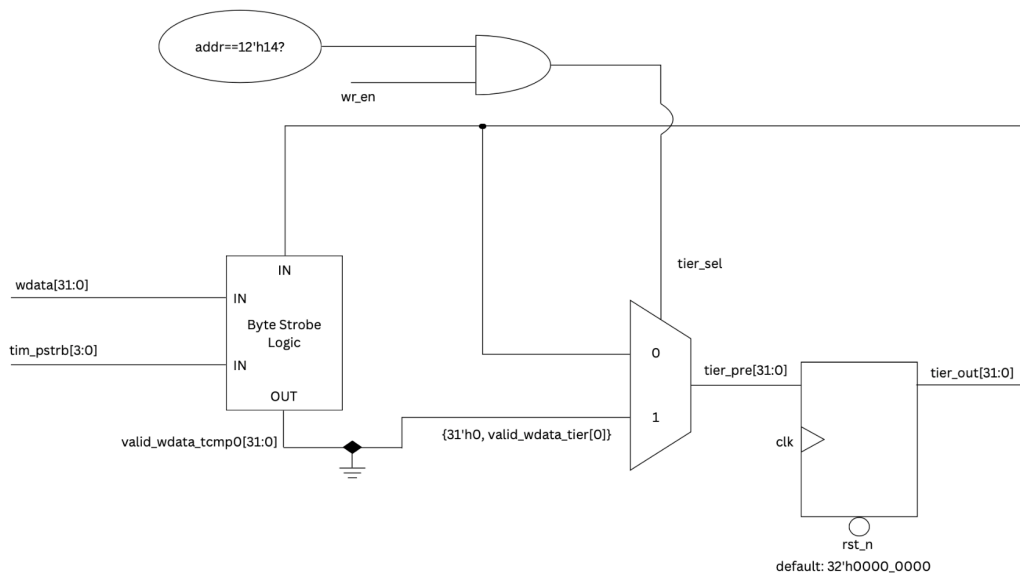
Module Description: TIER bit 0 (`int_en`) is the interrupt mask. Only bit 0 is writable; bits 31:1 are reserved/0. When `int_en = 1` the pending bit can propagate to `tim_int`; clearing it masks `tim_int` to 0 but does **not** clear the pending bit in TISR.

(Figure 12: Waveform for Timer Interrupt Enable Register)



Explain Waveform: With an interrupt already pending (`int_st_out_pre = 1`), writing 0x14 with bit 0 = 1 sets `tier_out[0]` one clock later; `tim_int` (= pending AND enable) then asserts. Writing bit 0 = 0 later would drop `tim_int` while the pending bit stays set.

(Figure 13: Logic Diagram of Timer Interrupt Enable Register)

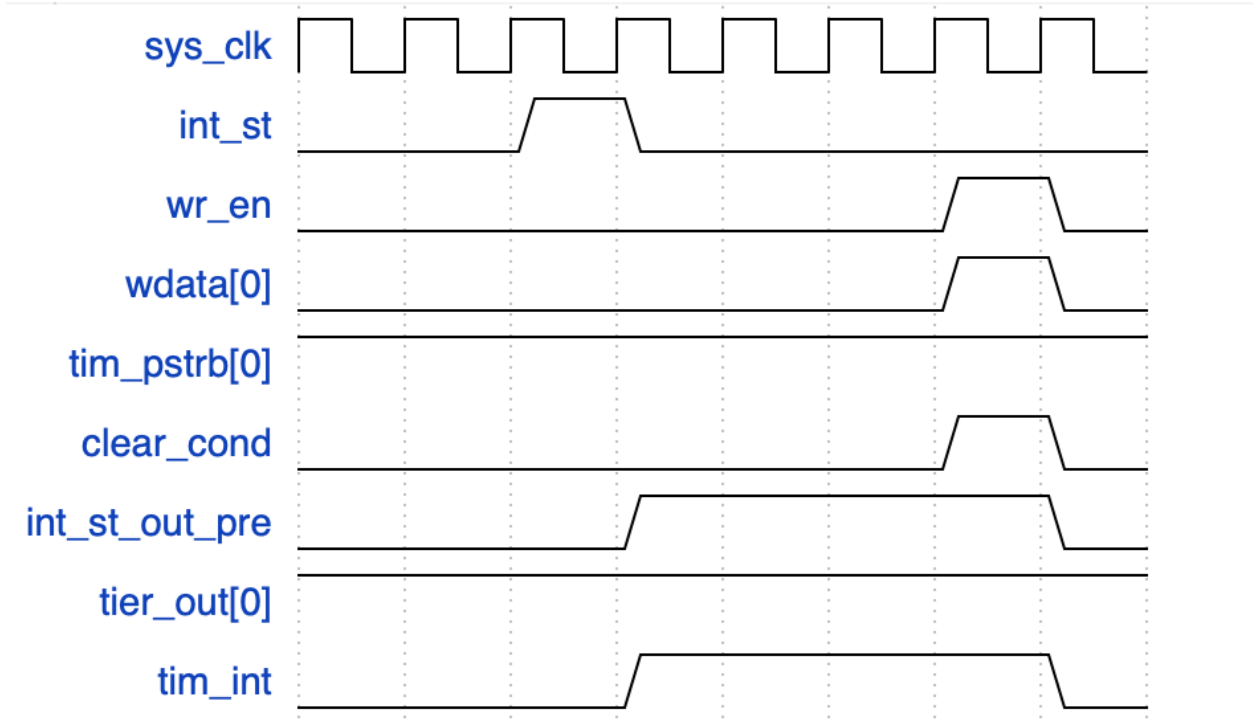


Explain Logic Diagram: $\text{addr} == 12'h14 \text{ AND } \text{wr_en} \rightarrow \text{tier_sel}$. $\text{tier_pre} = \text{tier_sel} ? \{31'h0, \text{valid_wdata_tier}[0]\} : \text{tier_out}$ forces bits 31:1 to 0 and only stores bit 0, registered each clock (async reset 0). $\text{tim_int} = \text{int_st_out_pre} \& \text{tier_out}[0]$ is formed in the TISR logic (3.1.7).

3.1.7. Timer Interrupt Status Register (0x18):

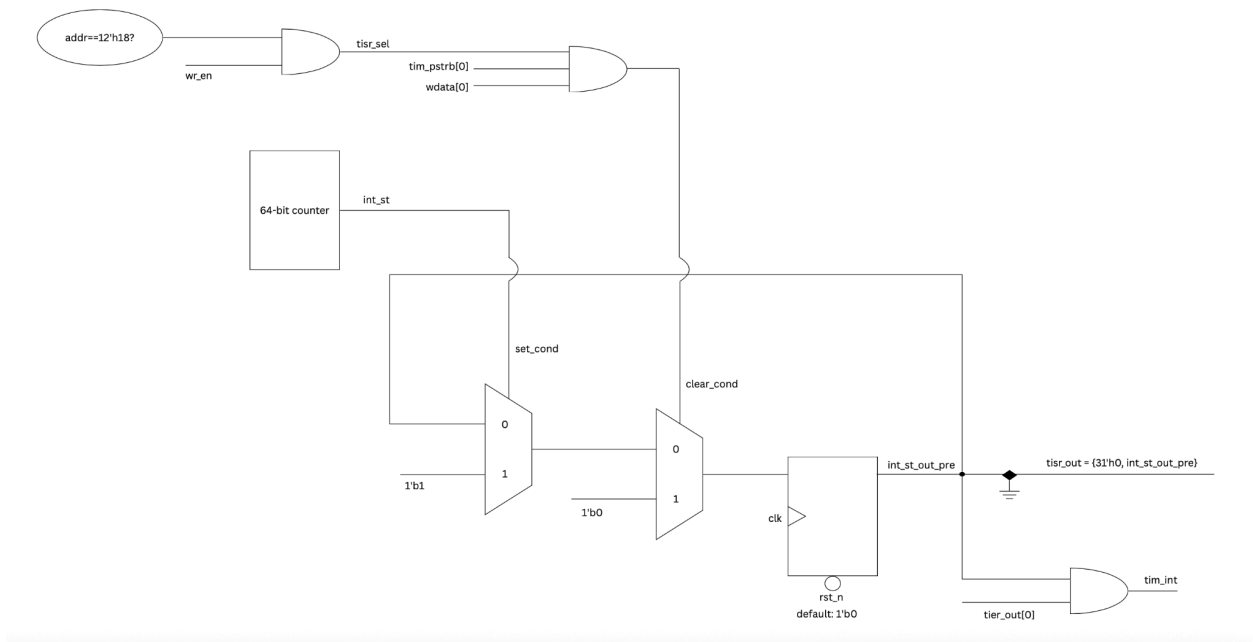
Module Description: TISR bit 0 (int_st) is the interrupt pending bit, type **W1C**. It is set by hardware when the counter reaches comp_val ($\text{set_cond} = \text{int_st}$ from the counter) and cleared by writing 1 (clear_cond). Clear has the highest priority; writing 0, or writing when the bit is already 0, has no effect. The counter keeps counting after the match.

(Figure 14: Waveform for Timer Interrupt Status Register)



Explain Waveform: A one-cycle **int_st** (count == comp_val) sets **int_st_out_pre**, which then stays High (sticky). With TIER enabled (**tier_out[0] = 1**), **tim_int** follows. A W1C write (**wr_en** & **tim_pstrb[0]** & **wdata[0]**) produces **clear_cond**, clearing the pending bit and **tim_int** one clock later.

(Figure 15: Logic Diagram of Timer Interrupt Status Register)

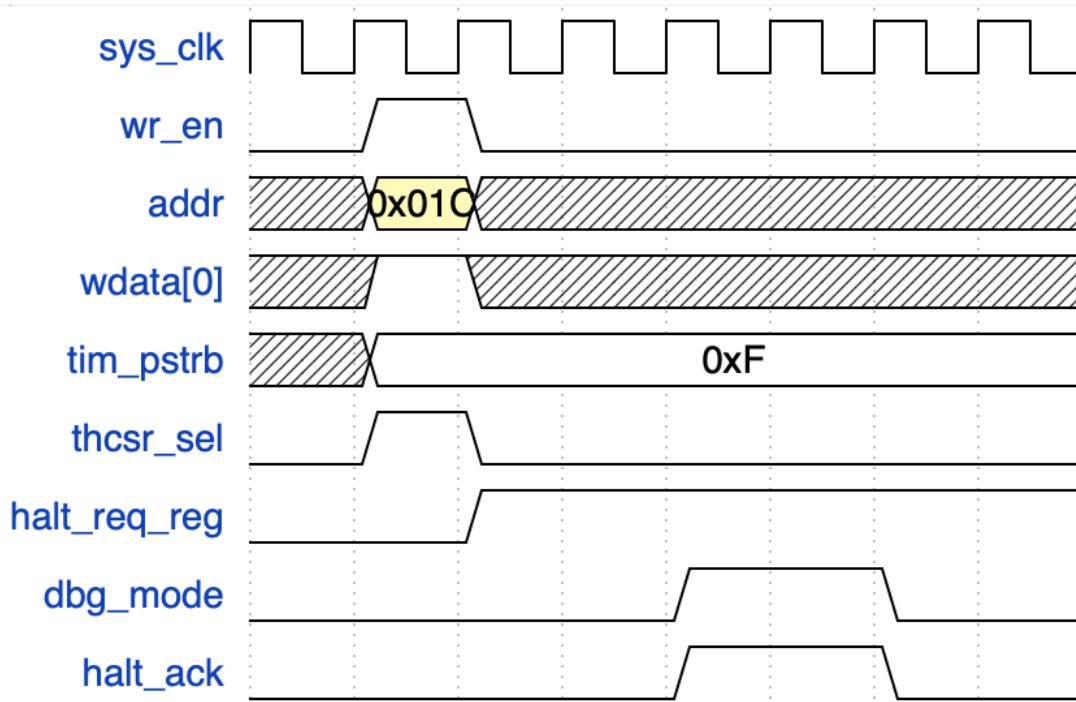


Explain Logic Diagram: $\text{tcsr_sel} = \text{addr} == 12'h18 \text{ AND } \text{wr_en}$; $\text{clear_cond} = \text{tcsr_sel} \text{ \& } \text{tim_pstrb}[0] \text{ \& } \text{wdata}[0]$; $\text{set_cond} = \text{int_st}$. Two priority muxes implement: $\text{clear_cond} \rightarrow 0$, else $\text{set_cond} \rightarrow 1$, else hold; registered (async reset 0) as int_st_out_pre . $\text{tcsr_out} = \{31'h0, \text{int_st_out_pre}\}$. The final AND, $\text{tim_int} = \text{int_st_out_pre} \text{ \& } \text{tier_out}[0]$, applies the TIER mask.

3.1.8. Timer Halt Control Status Register (0x1C):

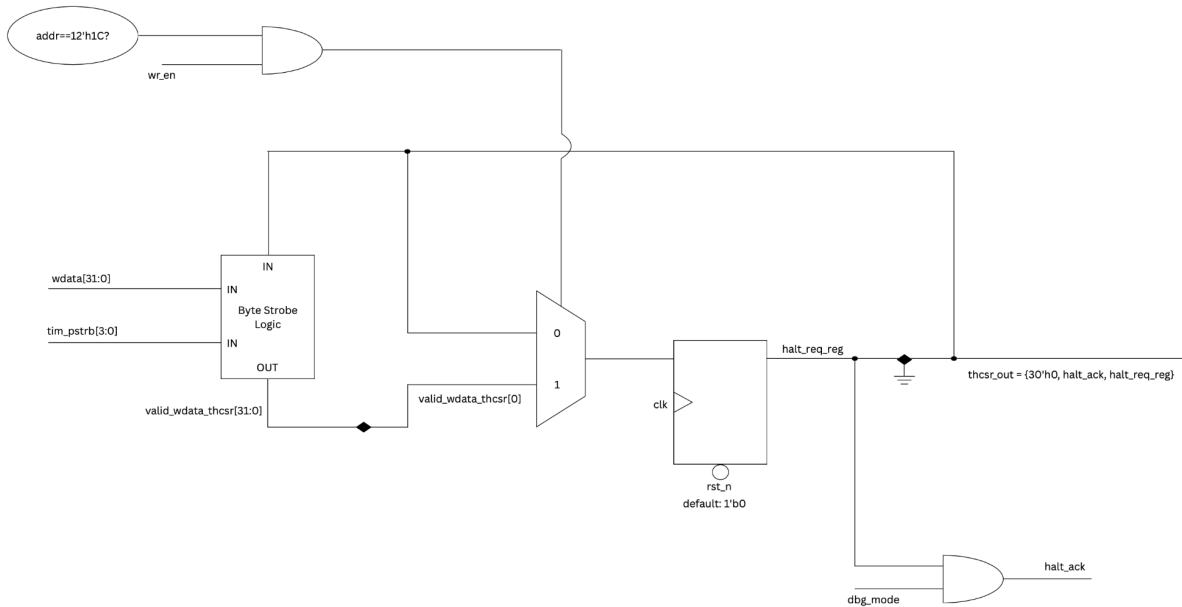
Module Description: THCSR bit 0 (halt_req , RW) requests a counter halt; bit 1 (halt_ack , RO) reports whether the counter is actually halted. The request is honored **only in debug mode**: $\text{halt_ack} = \text{halt_req_reg} \text{ \& } \text{dbg_mode}$. halt_ack stops the counter and freezes TDR mirroring.

(Figure 16: Waveform for Timer Halt Control Status Register)



Explain Waveform: Writing 0x1C with bit 0 = 1 sets halt_req_reg one clock later. halt_ack stays Low until dbg_mode also goes High; it asserts only while both are High, and drops as soon as dbg_mode deasserts even though the request is still latched.

(Figure 17: Logic Diagram of Timer Halt Control Status Register)

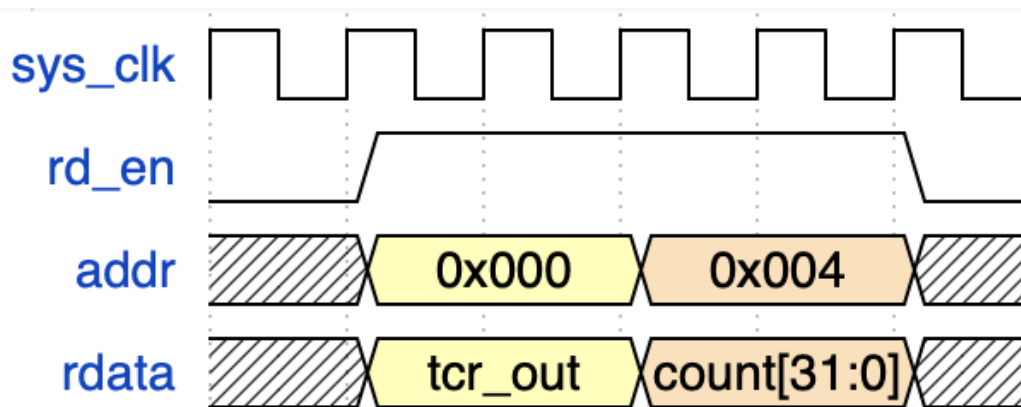


Explain Logic Diagram: $\text{addr} == 12'h1C \text{ AND } \text{wr_en} \rightarrow \text{thcsr_sel}$; the mux loads halt_req_reg from $\text{valid_wdata_thcsr}[0]$ on a write, else holds (async reset 0). $\text{halt_ack} = \text{halt_req_reg} \& \text{dbg_mode}$. $\text{thcsr_out} = \{30'h0, \text{halt_ack}, \text{halt_req_reg}\}$ exposes both bits for read-back.

3.1.9. Read Logic:

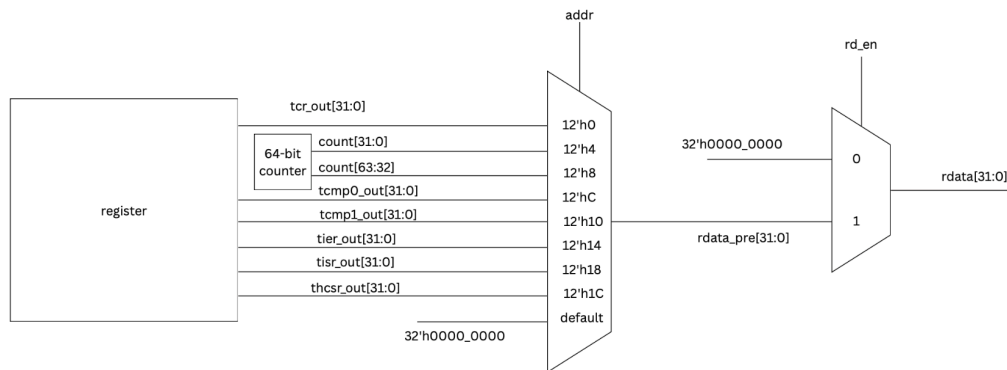
Module Description: Address-decoded read multiplexer. rdata_pre selects the addressed register; $\text{rdata} = \text{rd_en} ? \text{rdata_pre} : 32'h0$. Note that reads of $0x04/0x08$ return the **live counter** words ($\text{count}[31:0]/\text{count}[63:32]$), not a stored copy.

(Figure 18: Waveform for Read Logic)



Explain Waveform: While `rd_en` is High, `rdata` immediately reflects the register selected by `addr` (combinational mux): `0x000` → `tcr_out`, `0x004` → `count[31:0]`. When `rd_en` deasserts, `rdata` is forced to 0.

(Figure 19: Logic Diagram of Read Logic)

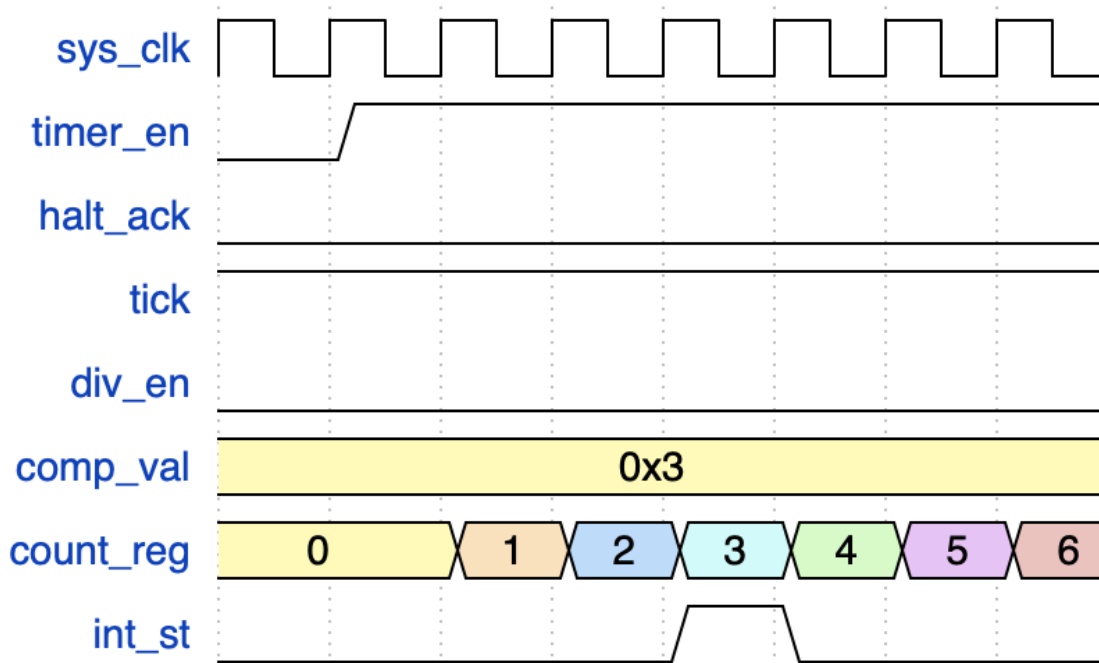


Explain Logic Diagram: A `case(addr)` mux maps each address to its source:
`0x0` → `tcr_out`, `0x4` → `count[31:0]`, `0x8` → `count[63:32]`, `0xC` → `tcmp0_out`,
`0x10` → `tcmp1_out`, `0x14` → `tier_out`, `0x18` → `tisr_out`, `0x1C` → `thcsr_out`, `default` → `32'h0`.
A final 2:1 mux gated by `rd_en` outputs `rdata_pre` or `32'h0000_0000`.

3.2. Counter:

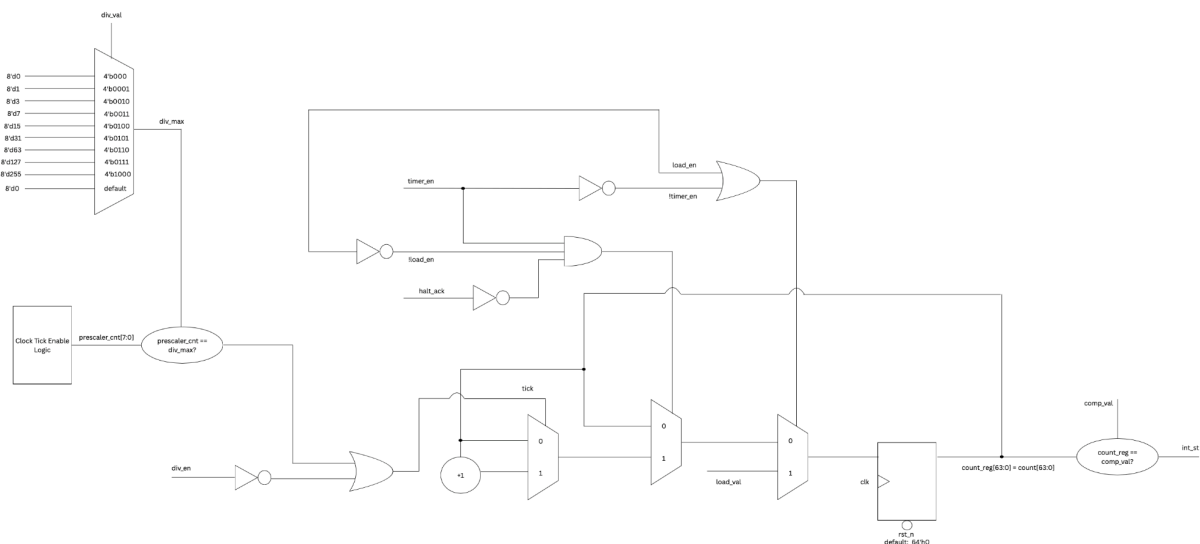
Module Description: A 64-bit up-counter. It loads `load_val` when `load_en` is High or whenever the timer is disabled (`!timer_en`); it increments by 1 on each tick while enabled and not halted (`timer_en && !halt_ack`); otherwise it holds. `int_st` asserts whenever `count_reg == comp_val`. The tick rate is set by the Clock Tick Enable Logic (3.2.1).

(Figure 20: Waveform for Counter)



Explain Waveform: With **div_en** = 0, **tick** is High every clock, so once **timer_en** goes High the counter increments each cycle: 0 → 1 → 2 → 3 → ... When **count_reg** reaches **comp_val** = 3, **int_st** pulses for that cycle; the counter does **not** stop — it continues to 4, 5, 6 ...

(Figure 21: Logic Diagram of Counter)



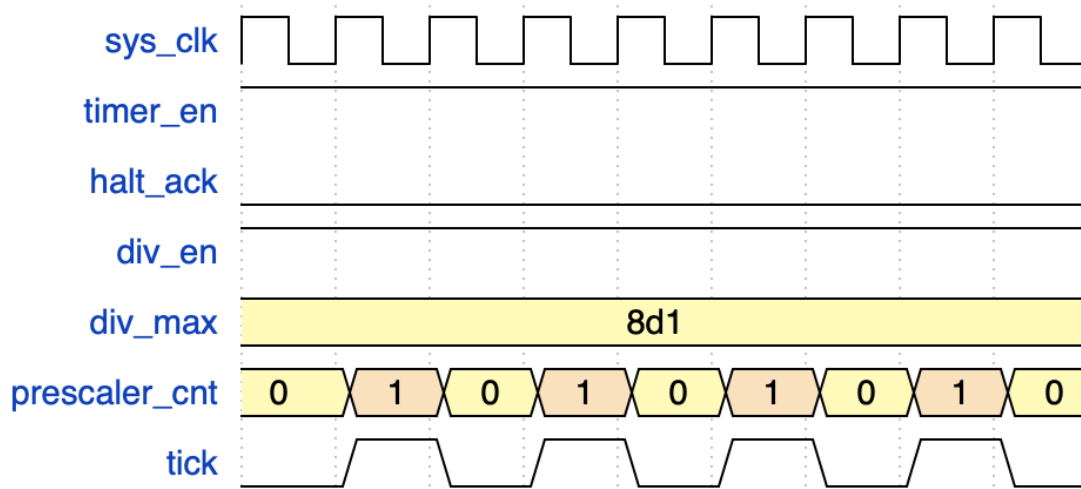
Explain Logic Diagram: The counter flop input is a priority mux: **load_en** || **!timer_en** → **load_val**; else **timer_en** && **!halt_ack** && **!load_en** && **tick** → **count_reg** + 1; else hold. **halt_ack** is inverted and ANDed with

timer_en to form the count-enable; !timer_en (via the OR with load_en) forces the load path. The comparator count_reg == comp_val drives int_st. The +1 adder feeds the increment leg of the mux gated by tick.

3.2.1. Clock Tick Enable Logic:

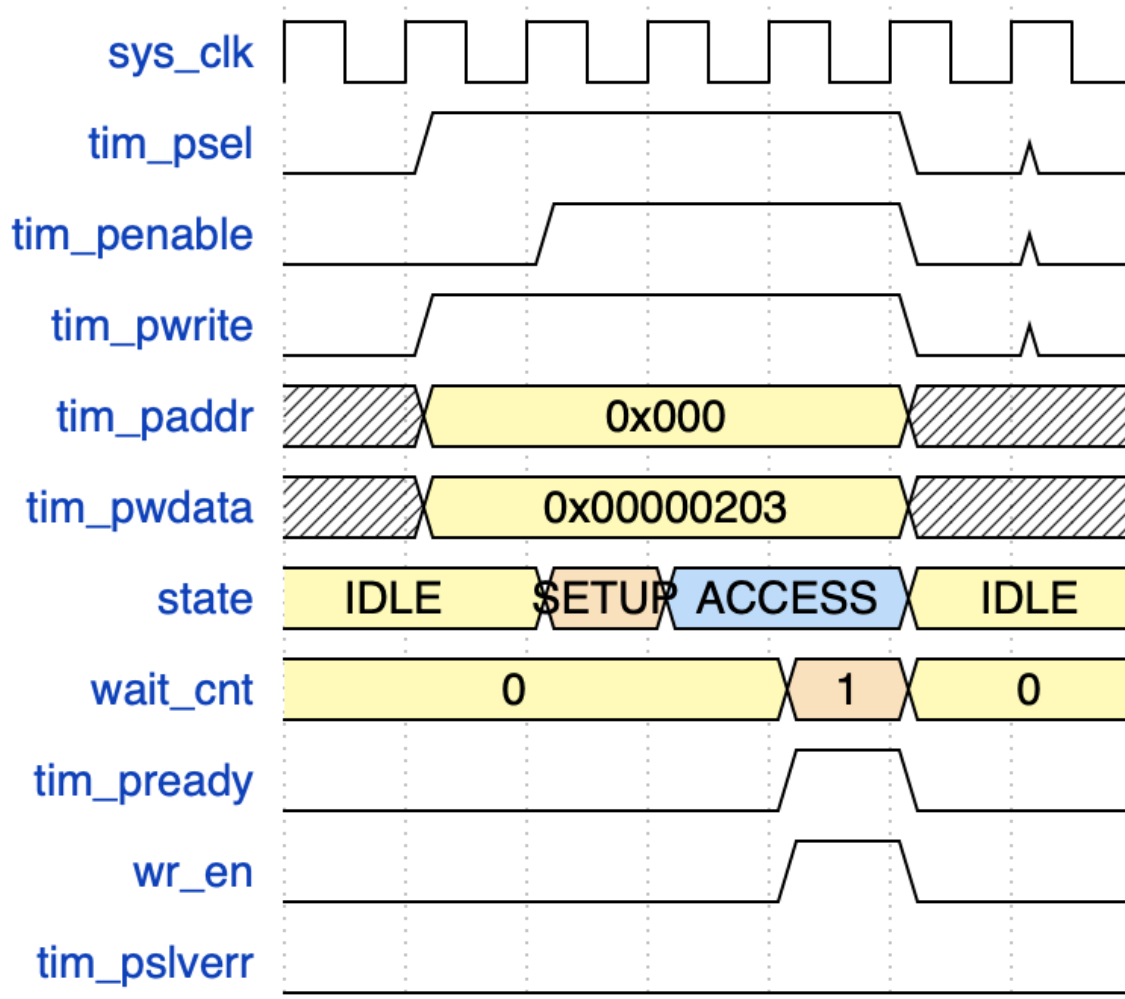
Module Description: A prescaler that paces the counter. If div_en = 0, tick is asserted every clock (full speed). If div_en = 1, prescaler_cnt counts 0 → div_max and tick asserts only when it reaches div_max, dividing the count rate. div_max is decoded from div_val (0,1,3,7,...,255 for ÷1,÷2,÷4,...,÷256). The prescaler resets on load_en || !timer_en and on each tick.

(Figure 22: Waveform for Clock Tick Enable Logic)



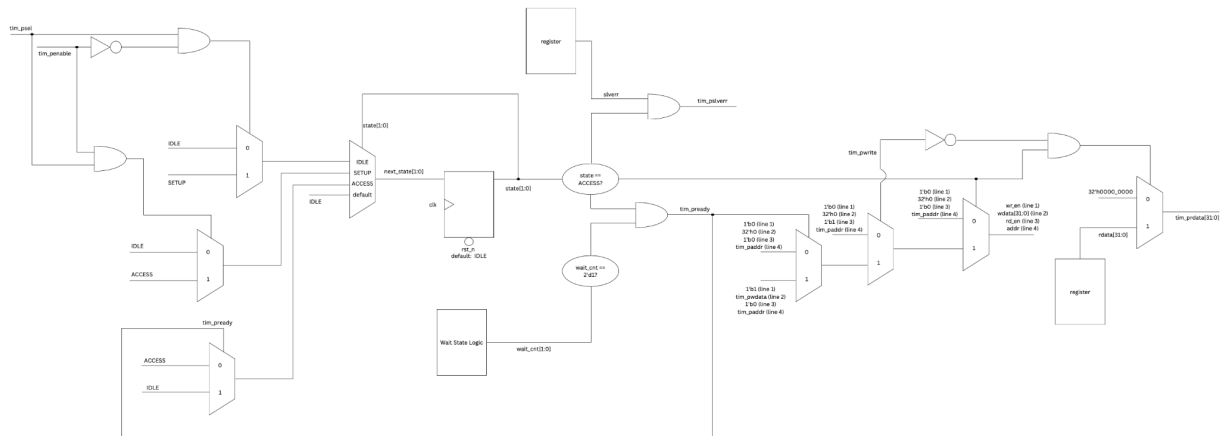
Explain Waveform: With div_max = 1, prescaler_cnt toggles 0 → 1 → 0 → 1 ...; tick = (prescaler_cnt == div_max) is High every second clock. The counter therefore advances once per two clocks (÷2). Larger div_val selects a larger div_max and a longer count before tick.

(Figure 23: Logic Diagram of Clock Tick Enable Logic)



Explain Waveform: The master asserts `tim_psel` with `tim_penable` Low → FSM moves IDLE → SETUP. Next cycle `tim_penable` goes High → SETUP → ACCESS. In ACCESS, `wait_cnt` counts 0 then 1; when `wait_cnt == 1`, `tim_pready` asserts and (because `tim_pwrite = 1`) `wr_en` pulses with `addr/wdata` valid, completing the write. The FSM returns to IDLE. No error here, so `tim_pslverr` stays Low.

(Figure 25: Logic Diagram of APB Slave)

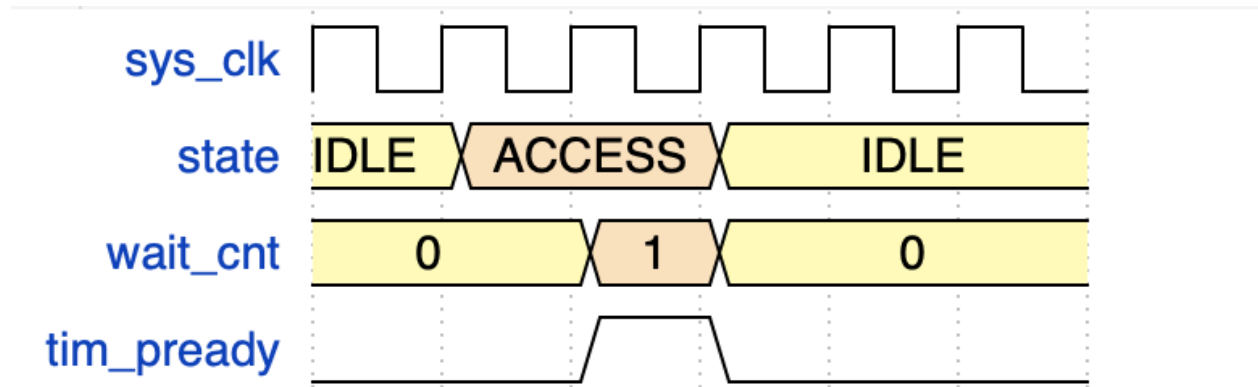


Explain Logic Diagram: Next-state logic: IDLE→SETUP on psel & !penable; SETUP→ACCESS on psel & penable; ACCESS→IDLE on tim_pready. The state flop is async-reset to IDLE. $\text{tim_pready} = (\text{state} == \text{ACCESS}) \& (\text{wait_cnt} == 1)$; $\text{tim_pslverr} = (\text{state} == \text{ACCESS}) \& \text{slverr}$. The output decode (the stacked muxes on the right) drives, in ACCESS: write path $\text{wr_en}=1$, $\text{wdata}=\text{tim_pdata}$ when $\text{tim_pwrite} \& \text{tim_pready}$; read path $\text{rd_en}=1$ throughout ACCESS; $\text{addr} = \text{tim_paddr}$. $\text{tim_prdata} = (\text{state} == \text{ACCESS} \& !\text{tim_pwrite}) ? \text{rdata} : 32'h0$.

3.3.1. Wait State Logic:

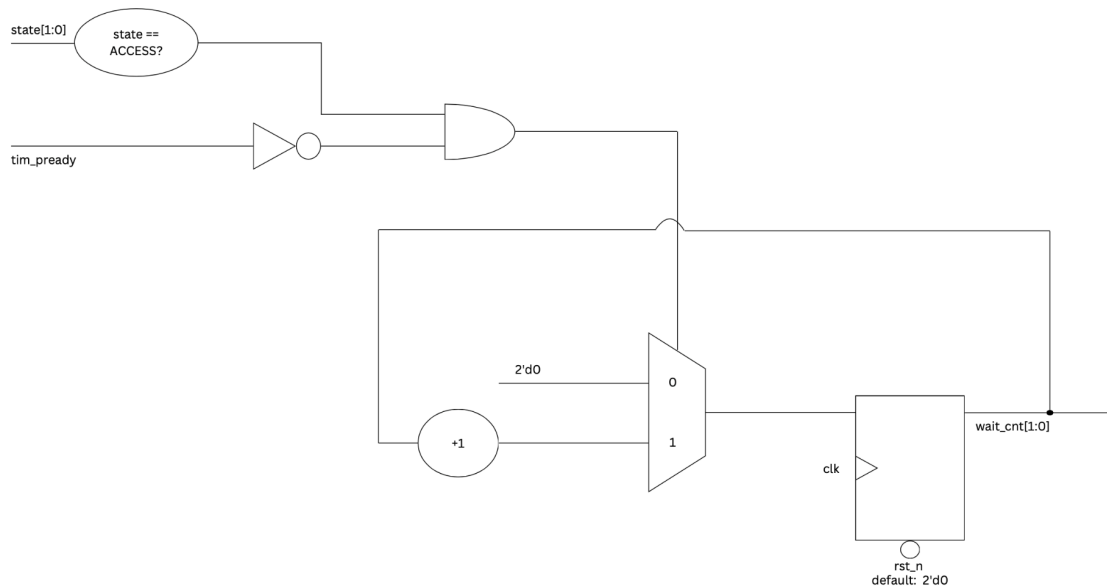
Module Description: Inserts a fixed one-cycle wait state during ACCESS. wait_cnt increments while in ACCESS and tim_pready is Low, and resets otherwise; tim_pready asserts when $\text{wait_cnt} == 1$, making ACCESS two cycles long (one wait cycle + one ready cycle).

(Figure 26: Waveform for Wait State Logic)



Explain Waveform: On entering ACCESS, wait_cnt = 0 and tim_pready = 0 (the wait cycle). Next clock wait_cnt = 1, so tim_pready asserts (the ready cycle); the transfer completes and the FSM leaves ACCESS, resetting wait_cnt to 0.

(Figure 27: Logic Diagram of Wait State Logic)



Explain Logic Diagram: The mux select is (state == ACCESS) & !tim_pready: when true the flop loads wait_cnt + 1 (the +1 adder leg), otherwise it loads 2'd0. The flop is async-reset to 2'd0. The combinational comparator state == ACCESS? AND wait_cnt == 1? (in the parent APB Slave) generates tim_pready, which both ends the count and triggers the ACCESS→IDLE transition.

4. History:

Date	Version	Description	Author
11/05/2026	1.0	Newly created	Do Luong Minh Hoang
12/05/2026	1.1	Modified	Do Luong Minh Hoang
19/05/2026	1.2	Modified	Do Luong Minh Hoang
21/06/2026	1.3	Modified	Do Luong Minh Hoang
22/06/2026	1.4	Modified	Do Luong Minh Hoang